

基于 Verilog 的 I2C 总线控制器设计与验证

本文为本科毕业论文初稿。封面、任务书、姓名、学号、学院、专业、指导教师、日期等信息请在最终排版阶段按学校模板补充。本文中的项目事实主要依据 `test/project-report.md` 与 项目/`verilog-i2c-master` 的 README、RTL 源码和测试平台文件整理；未实际运行仿真或综合的部分均以“待补充”标明。

摘要

I2C 总线因连线简单、设备扩展方便、协议开销较低，被广泛用于 FPGA 板级外设配置、传感器访问、EEPROM 读写、时钟芯片初始化和片上系统低速控制通道。针对 FPGA 系统中 I2C 外设访问对可复用硬件 IP 的需求，本文以 Verilog I2C interface 项目为研究对象，设计并分析一种基于 Verilog HDL 的 I2C 总线控制器及其验证方法。该设计以 I2C 主机核心和 I2C 从机核心为基础，采用 AXI Stream 风格接口组织命令流和数据流，并通过 AXI Lite、Wishbone 等总线封装方式实现与处理器或片上总线系统的连接。同时，系统提供基于 ROM 表的 I2C 初始化模块，可用于无通用处理器参与的上电外设配置。

本文首先分析 I2C 协议的起始条件、停止条件、地址传输、读写控制、ACK/NACK 应答、开漏总线和时钟拉伸等关键机制；随后从需求分析、总体结构、模块划分、状态机设计、接口设计和寄存器映射等方面展开系统设计。主机模块采用命令层状态机与位级物理层状态机分层实现，完成 start、repeated start、读位、写位和 stop 等操作；从机模块通过地址匹配、输入滤波、AXI Stream 数据转换和时钟拉伸机制实现 I2C 从设备功能；AXI Lite 与 Wishbone 封装模块通过状态寄存器、命令寄存器、数据寄存器和分频寄存器完成上层控制。最后，本文结合 MyHDL 与 Icarus Verilog 协同仿真平台，给出测试思路、验证场景和结果记录方式。

研究表明，该类 I2C 控制器具有模块化程度高、接口适配性强、可综合性好和便于 FPGA 系统集成等特点。由于当前材料尚未包含本机仿真日志、综合报告和板级实测数据，本文将相关实测结果列为后续补充项。整体而言，本文工作可为 FPGA 中低速外设控制器设计、标准总线封装和硬件模块验证提供参考。

关键词：I2C 总线，Verilog，FPGA，AXI Lite，Wishbone，协同仿真

Abstract

The I2C bus is widely used in FPGA-based peripheral configuration, sensor access, EEPROM read and write operations, clock chip initialization, and low-speed control channels in system-on-chip designs because of its simple wiring, flexible device expansion, and low protocol overhead. To meet the demand for reusable hardware IP for I2C peripheral access in FPGA systems, this thesis studies a Verilog-based I2C bus controller and its verification method based on the Verilog I2C interface project. The design is built around an I2C master core and an I2C slave core. AXI Stream-style interfaces are used to organize command and data streams, while AXI Lite and Wishbone wrappers are provided

to connect the controller to processors or on-chip bus systems. In addition, an ROM-table-driven I2C initialization module is provided for power-on peripheral configuration without the participation of a general-purpose processor.

This thesis first analyzes key mechanisms of the I2C protocol, including start and stop conditions, address transmission, read/write control, ACK/NACK responses, open-drain bus behavior, and clock stretching. Then the system is discussed from the perspectives of requirement analysis, overall architecture, module partitioning, state machine design, interface design, and register mapping. The master module adopts a layered structure consisting of a command-level state machine and a bit-level physical state machine to implement start, repeated start, bit read, bit write, and stop operations. The slave module implements I2C slave functionality through address matching, input filtering, AXI Stream data conversion, and clock stretching. The AXI Lite and Wishbone wrapper modules expose status, command, data, and prescale registers for upper-layer control. Finally, a co-simulation verification platform based on MyHDL and Icarus Verilog is introduced, including test scenarios and result recording methods.

The analysis shows that this type of I2C controller has high modularity, strong interface adaptability, good synthesizability, and convenient integration into FPGA systems. Since the current materials do not include local simulation logs, synthesis reports, or board-level measurement data, these experimental results are marked as items to be supplemented in future work. Overall, this thesis can serve as a reference for FPGA low-speed peripheral controller design, standard bus wrapping, and hardware module verification.

Key words: I2C bus, Verilog, FPGA, AXI Lite, Wishbone, co-simulation

1 绪论

1.1 研究背景与意义

随着嵌入式系统、工业控制、通信设备和智能硬件的快速发展，系统中需要连接和管理的低速外设数量不断增加。常见外设包括 EEPROM、RTC、温湿度传感器、显示控制芯片、时钟发生器、电源管理芯片和板级监控芯片等。这些器件往往不需要高速并行数据传输，却要求接口简单、引脚占用少、控制方式清晰。I2C 总线正是在这种需求下被广泛采用的一种串行通信方式。

I2C 总线由 SCL 和 SDA 两根信号线组成，可通过 7 位或 10 位地址区分多个从设备。由于其采用开漏输出和上拉电阻结构，多个器件可以共享同一总线，并通过起始条件、停止条件、地址字段、读写位和应答位完成通信控制。与 SPI 相比，I2C 在连接多个低速外设时可以节省片选线；与 UART 相比，I2C 支持总线寻址和主从设备结构，更适合板级多外设管理。因此，在 FPGA 系统中实现一个稳定、可复用、易集成的 I2C 控制器具有实际工程意义。

在 FPGA 应用中，I2C 控制可以通过处理器软件驱动外设控制器完成，也可以由硬件逻辑直接实现。前者适合带软核或硬核处理器的 SoC 系统，后者适合对启动配

置、实时响应或资源隔离有要求的纯逻辑系统。硬件 I2C 控制器不仅可以降低软件参与程度，还能在系统上电早期完成外设初始化，例如配置 PLL、时钟复用器、抖动衰减器或传感器工作模式。若控制器进一步封装为 AXI Lite 或 Wishbone 等片上总线接口，则可被嵌入式处理器通过寄存器访问，从而提升系统集成灵活性。

本文研究的 Verilog I2C interface 项目提供 I2C 主机、I2C 从机、AXI Lite 封装、Wishbone 封装、AXI Stream FIFO 和上电初始化模板等模块，具有较完整的硬件 IP 结构。围绕该项目展开毕业论文写作，可以系统分析 I2C 协议硬件实现、标准片上总线封装、状态机设计、流式接口设计和协同仿真验证方法，有助于加深对数字系统设计流程的理解。

1.2 国内外研究现状

I2C 协议由 Philips（现 NXP）提出，后续不断扩展并形成较完善的规范体系。NXP 发布的 UM10204 I2C-bus specification and user manual 是 I2C 协议设计的重要依据，其中规定了总线电气特性、数据传输格式、时钟同步、仲裁、ACK/NACK 应答和不同速率模式等内容。对于硬件控制器设计而言，该规范为起止条件识别、数据采样时序、时钟拉伸和多设备总线共享提供了权威参考。

在国外研究和工程实践中，I2C 控制器通常以 FPGA IP、SoC 外设或开源硬件模块形式存在。AMD/Xilinx 的 AXI IIC Bus Interface IP 将 I2C 控制器封装为 AXI 接口外设，支持处理器通过寄存器对 I2C 总线进行控制，体现了工业 FPGA 平台中常见的 IP 集成方式。OpenCores 中的 I2C controller core 则是典型的开源 Wishbone-I2C 控制器实现，其设计思路对 Wishbone 总线封装、寄存器控制和开源 IP 复用具有参考价值。

在国内研究中，围绕“基于 FPGA 的 I2C 总线控制器设计”“基于 Verilog 的 I2C 协议实现”“I2C 总线接口设计与仿真”等方向已有较多课程设计、工程论文和学位论文。相关工作通常关注 I2C 总线时序、有限状态机划分、FPGA 实现、ModelSim 仿真和外设读写验证。与单一 I2C 主机控制器相比，本文所分析的项目具有主从双向支持、多种片上总线封装和 MyHDL 协同仿真环境等特点，因此更适合作为完整硬件 IP 设计与验证案例。

总体来看，I2C 控制器本身并非新协议研究问题，其重点在于如何在特定 FPGA 系统中实现可综合、可复用、可验证和易集成的硬件模块。本文将从模块化设计角度出发，对项目中的主机核心、从机核心、总线封装和测试平台进行系统整理。

1.3 研究内容与技术路线

本文主要研究内容包括以下几个方面：

1. I2C 协议原理分析。梳理 I2C 总线的物理连接方式、起止条件、地址传输、读写方向、ACK/NACK 应答、时钟拉伸和总线状态判断，为硬件实现提供协议基础。
2. 系统需求分析。根据 Verilog I2C interface 项目资料，总结系统应支持的主机通信、从机通信、AXI Lite 封装、Wishbone 封装、初始化序列和仿真验证等功能。

3. 总体结构设计。分析项目中 RTL 与测试平台的目录结构，给出 I2C 核心层、流式接口层、总线封装层和验证层之间的关系。
4. 关键模块设计。重点分析 `i2c_master`、`i2c_slave`、`i2c_master_axil`、`i2c_master_wbs_8`、`i2c_init` 和 `axis_fifo` 等模块的功能、接口、状态机和数据流。
5. 仿真验证方法。结合 MyHDL 和 Icarus Verilog 协同仿真环境，讨论主机读写、从机读写、FIFO 缓冲、寄存器访问、ACK 检测、时钟拉伸等测试场景。
6. 总结不足与改进方向。针对当前材料中缺少综合结果、板级验证和完整测试日志的问题，提出后续完善方案。

本文技术路线为：首先依据 I2C 协议规范和项目 README 明确设计目标；然后读取项目报告和核心 RTL 文件，建立模块结构和数据流模型；接着围绕主从控制器状态机和总线封装寄存器展开详细设计分析；最后结合项目测试平台提出验证方案，并对设计特点与不足进行总结。

1.4 论文组织结构

本文共分为七章。第 1 章为绪论，介绍研究背景、研究意义、国内外现状、研究内容和论文结构。第 2 章介绍 I2C 协议、Verilog HDL、FPGA、AXI Stream、AXI Lite、Wishbone 和协同仿真等相关技术。第 3 章进行系统需求分析，明确功能需求、非功能需求和设计约束。第 4 章给出系统总体设计，包括总体架构、模块划分、数据流和接口关系。第 5 章详细分析关键模块的实现，包括 I2C 主机、从机、总线封装、初始化模块和 FIFO 模块。第 6 章介绍测试与验证方案，并整理当前验证结果和待补充实验。第 7 章总结全文工作，分析不足并提出后续展望。

2 相关技术与理论基础

2.1 I2C 总线协议

I2C 是一种同步串行总线，典型连接方式包括一根串行时钟线 SCL 和一根串行数据线 SDA。总线空闲时，SCL 与 SDA 均保持高电平。主设备通过控制 SDA 在 SCL 高电平期间由高到低变化产生起始条件，通过 SDA 在 SCL 高电平期间由低到高变化产生停止条件。起始条件之后，总线进入活动状态；停止条件之后，总线回到空闲状态。

I2C 传输通常以 8 位为单位进行。主设备首先发送 7 位从设备地址和 1 位读写方向位，其中读写位为 0 表示写操作，为 1 表示读操作。每发送 8 位数据后，接收方需要在第 9 个时钟周期给出 ACK 或 NACK。ACK 通常表现为接收方拉低 SDA，NACK 则表现为释放 SDA 保持高电平。通过 ACK/NACK，发送方可以判断对方是否正确接收数据，或判断读操作是否需要结束。

I2C 的一个重要特性是开漏总线结构。设备不能主动驱动高电平，只能拉低总线或释放总线。因此，当多个设备共享总线时，只要任一设备拉低信号线，总线就表现为低电平。这一特性支持多设备共享、仲裁和时钟拉伸。时钟拉伸是指从设备在无法及时提供或接收数据时，可以拉低 SCL 暂停传输，直到准备好后释放 SCL。

在硬件实现中，I2C 控制器需要准确识别 start、stop、SCL 上升沿、SCL 下降沿、

SDA 边沿以及 ACK/NACK 状态。同时，需要避免将输出信号直接反馈到输入端，以免破坏开漏和时钟拉伸机制。因此，本文项目采用 `scl_i/scl_o/scl_t` 和 `sda_i/sda_o/sda_t` 的接口形式，分别表示输入采样、输出值和三态控制。

2.2 Verilog HDL 与 FPGA 设计

Verilog HDL 是数字逻辑设计中常用的硬件描述语言，能够描述组合逻辑、时序逻辑、状态机、模块层次和可综合电路结构。FPGA 则提供可编程逻辑资源、触发器、片上存储器、时钟管理资源和高速 I/O 等硬件基础，使设计者能够在现场实现和验证数字系统。

在 FPGA 设计流程中，通常先使用 Verilog 编写 RTL 模块，然后进行功能仿真、综合、布局布线、时序分析和板级验证。对于通信控制器类模块，状态机设计是核心内容。良好的状态机应具有清晰的状态定义、确定的状态转移条件、完整的复位行为和可靠的输出控制。I2C 控制器涉及位级时序控制、字节级事务管理和上层接口握手，因此适合采用多层状态机组织逻辑。

本文项目的 RTL 源码采用 Verilog 2001 风格，主要模块均以可综合形式编写。模块中大量使用寄存器保存状态、地址、数据、计数器和握手标志，并通过组合逻辑计算下一状态，通过时钟边沿更新寄存器。这种写法符合常见 FPGA 综合工具的处理方式。

2.3 AXI Stream 与 AXI Lite 接口

AXI 是 ARM AMBA 总线规范中的重要组成。AXI Stream 面向连续数据流传输，核心握手机制为 `tvalid` 和 `tready`：发送端在数据有效时置位 `tvalid`，接收端在能够接收时置位 `tready`，二者同时有效时完成一次传输。`tdata` 表示数据，`tlast` 可用于标记帧或事务结束。

本文项目使用 AXI Stream 风格接口组织 I2C 主机命令、写数据和读数据。这种方式的优点是控制逻辑与数据缓冲可以解耦，模块可以与 FIFO、DMA、状态机或其他流式模块灵活连接。例如，在 I2C 多字节写操作中，`tlast` 可用于标记最后一个写入字节；在读操作中，`m_axis_data_tlast` 可用于标记读事务结束。

AXI Lite 则是 AXI 的简化寄存器访问接口，适合低带宽控制外设。AXI Lite 包括写地址、写数据、写响应、读地址和读数据通道，通常用于处理器访问外设寄存器。本文项目中的 `i2c_master_axil` 模块将 I2C 主机封装为 AXI Lite 从设备，提供状态、命令、数据和分频寄存器，便于软件通过寄存器发起 I2C 操作。

2.4 Wishbone 总线接口

Wishbone 是开源硬件领域常见的片上总线规范，常用于开源 SoC 和 FPGA 外设连接。Wishbone 接口通常包含地址、数据输入、数据输出、写使能、周期、选通和应答等信号。与 AXI Lite 类似，Wishbone 也适合寄存器型外设控制。

本文项目提供 `i2c_master_wbs_8` 和 `i2c_master_wbs_16` 两种 Wishbone 主机封装，分别适配 8 位和 16 位数据宽度。以 8 位封装为例，寄存器包括 Status、FIFO Status、Cmd Addr、Command、Data、Prescale Low 和 Prescale High 等。

通过 Wishbone 封装，I2C 控制器可以集成到采用 Wishbone 总线的开源 SoC 系统中。

2.5 MyHDL 与 Icarus Verilog 协同仿真

硬件设计验证不仅需要检查 RTL 语法和综合结果，还需要通过仿真验证功能正确性。Icarus Verilog 是开源 Verilog 编译和仿真工具，可将 Verilog 测试平台和 DUT 编译为可执行仿真文件。MyHDL 是基于 Python 的硬件建模和验证工具，可以用于构建高层行为模型、总线功能模型和测试激励。

本文项目的测试平台采用 MyHDL 与 Icarus Verilog 协同仿真方式。Python 测试脚本中创建 AXI Stream source/sink、I2C 存储器模型、AXI Lite BFM 和 Wishbone 模型，再通过 Icarus Verilog 编译 Verilog DUT，并使用 `vvp -m myhdl` 加载 MyHDL VPI 实现联动。该方法既能利用 Verilog RTL 的真实实现，又能利用 Python 语言编写灵活测试激励，有利于提高验证效率。

3 系统需求分析

3.1 设计目标

本文设计目标是实现并分析一种可复用的 I2C 总线控制器硬件 IP，使其能够在 FPGA 或 SoC 系统中完成 I2C 外设访问。结合项目资料，可将目标细化为以下几点：

1. 实现 I2C 主机控制器，支持基本读写、多字节写、起始条件、重复起始条件和停止条件。
2. 实现 I2C 从机控制器，支持地址匹配、读写方向判断、数据收发和总线状态输出。
3. 提供 AXI Stream 风格接口，便于命令和数据流与 FIFO 或上层逻辑连接。
4. 提供 AXI Lite 封装，使处理器能够通过寄存器控制 I2C 主机。
5. 提供 Wishbone 封装，使控制器适配开源 SoC 或 Wishbone 总线系统。
6. 提供 I2C 初始化模块，使 FPGA 可在上电后自动配置一个或多个 I2C 外设。
7. 提供可运行的仿真测试平台，支持对主机、从机和总线封装模块进行功能验证。

3.2 功能需求

3.2.1 I2C 主机功能需求

I2C 主机模块需要根据上层命令驱动 SCL 和 SDA，完成总线传输。其功能需求包括：

- 接收 7 位目标地址；
- 支持 read、write、write multiple 和 stop 命令；
- 支持 start 和 repeated start；
- 支持多字节写入，并通过 `tlast` 标记结束；
- 支持读数据输出，并在需要时标记最后一个字节；
- 检测从机 ACK，未检测到 ACK 时输出 `missed_ack` 状态；

- 输出 `busy`、`bus_control` 和 `bus_active` 等状态；
- 通过 `prescale` 配置 I2C 时钟速率。

3.2.2 I2C 从机功能需求

I2C 从机模块需要响应外部主机访问，并将 I2C 总线事务转换为内部流式数据。其功能需求包括：

- 支持 7 位设备地址配置；
- 支持地址掩码配置，以实现单地址或部分地址匹配；
- 检测 I2C 起始条件和停止条件；
- 支持外部主机写入数据，并将数据输出到 AXI Stream 接口；
- 支持外部主机读取数据，并从 AXI Stream 输入接口获取待发送字节；
- 在数据未准备好时支持 SCL 时钟拉伸；
- 输出当前总线地址、是否被寻址、总线是否活动和模块是否忙碌等状态。

3.2.3 总线封装功能需求

AXI Lite 和 Wishbone 封装模块用于将 I2C 控制器接入上层系统。封装模块应满足以下需求：

- 将命令、数据、分频和状态映射为寄存器；
- 提供 FIFO 空、满、溢出等状态；
- 支持软件或总线主设备写入命令和数据；
- 支持软件或总线主设备读取接收数据和错误状态；
- 保持寄存器访问与 I2C 总线传输之间的时序解耦。

3.2.4 初始化功能需求

`i2c_init` 模块面向上电自动配置场景，需求包括：

- 使用内部 ROM 表保存初始化命令；
- 支持单设备初始化；
- 支持多个设备执行相同初始化序列；
- 支持写地址、写数据、延时和停止命令；
- 输出符合 I2C 主机命令接口的数据流。

3.3 非功能需求

1. 可综合性：所有 RTL 核心模块应能被 FPGA 综合工具识别和综合。
2. 可复用性：模块接口应标准化，便于在不同 FPGA 工程中复用。
3. 可配置性：分频、FIFO 深度、地址掩码等参数应可配置。
4. 可验证性：应提供测试平台和行为模型，便于自动化仿真。
5. 兼容性：应适配 AXI Stream、AXI Lite 和 Wishbone 等多种接口。
6. 可靠性：应正确处理 ACK/NACK、总线活动状态、时钟拉伸和 FIFO 状态。

3.4 设计约束

本文项目存在以下约束：

1. 代码语言为 Verilog 2001，设计风格应符合 RTL 综合习惯。
2. I2C 总线为开漏结构，SCL/SDA 连接需通过三态或线与逻辑实现。
3. `prescale` 决定 I2C 时钟速率，其配置应与系统输入时钟和目标 I2C 速率匹配。
4. MyHDL 协同仿真依赖 Python、MyHDL、Icarus Verilog 和 `myhdl.vpi`。
5. 当前材料未包含综合报告、板级验证截图和完整仿真日志，相关数据不能凭空编写。

4 系统设计

4.1 总体架构

系统总体架构可分为四层：协议核心层、流式接口层、总线封装层和验证层。

协议核心层包括 `i2c_master` 和 `i2c_slave`。主机核心负责主动发起 I2C 事务，从机核心负责响应外部 I2C 主机访问。二者均直接连接 I2C 物理信号，并负责起止条件、地址、读写、ACK/NACK 和总线状态处理。

流式接口层主要由 AXI Stream 风格接口和 `axis_fifo` 组成。主机命令、写数据、读数据以及从机数据收发均可通过 `ready/valid/tlast` 握手机制组织。FIFO 模块用于缓冲命令和数据，降低上层访问与底层总线时序之间的耦合。

总线封装层包括 AXI Lite 和 Wishbone 封装模块。该层将流式命令和数据转换为寄存器访问，使处理器或片上总线主设备能够通过读写寄存器控制 I2C 传输。

验证层由 `tb/` 目录下的 Python 与 Verilog 文件构成，包括 I2C 行为模型、AXI Stream 端点、AXI Lite BFM、Wishbone 模型和各模块测试脚本。验证层用于生成激励、驱动 DUT、检查返回数据并记录仿真波形。

4.2 模块划分

系统主要模块划分如表 4.1 所示。

表 4.1 系统主要模块划分

模块	文件	功能
AXI Stream FIFO	<code>axis_fifo.v</code>	提供参数化流式数据缓冲
I2C 初始化模块	<code>i2c_init.v</code>	解释 ROM 初始化表并输出 I2C 命令
I2C 主机核心	<code>i2c_master.v</code>	生成 I2C 主机读写时序
I2C 主机 AXI Lite 封装	<code>i2c_master_axil.v</code>	提供 32 位 AXI Lite 寄存器接口
I2C 主机 Wishbone 封装	<code>i2c_master_wbs_8.v</code> 、 <code>i2c_master_wbs_16.v</code>	提供 8 位和 16 位 Wishbone 接口

模块	文件	功能
I2C 从机核心	<code>i2c_slave.v</code>	将 I2C 从机事务转换为 AXI Stream 数据
I2C 从机 AXI Lite master 封装	<code>i2c_slave_axil_master.v</code>	将 I2C 从机访问映射为 AXI Lite master 操作
I2C 从机 Wishbone master 封装	<code>i2c_slave_wbm.v</code>	将 I2C 从机访问映射为 Wishbone master 操作

4.3 数据流设计

主机写数据流为：上层逻辑或处理器写入命令和数据，命令进入主机命令接口，数据进入写数据接口；主机模块产生 I2C 起始条件，发送地址和写方向位，随后逐字节发送数据并检测 ACK；若命令要求停止，则发送停止条件。

主机读数据流为：上层写入读命令，主机模块产生起始条件并发送地址和读方向位；从机应答后，主机逐位采样 SDA，组成 8 位数据，通过读数据输出接口返回上层；若为最后一个字节，则产生 NACK 或 stop，并通过 `tlast` 标记。

从机写数据流为：外部主机发送地址和写方向位，从机地址匹配后接收后续字节；接收的数据被转换为 AXI Stream 输出。为了判断最后一个写入字节，从机模块对输出数据进行一个字节时间的延迟，以便结合 stop 条件设置 `tlast`。

从机读数据流为：外部主机发送地址和读方向位，从机地址匹配后从 AXI Stream 输入侧读取待发送数据；若本地数据未准备好，从机拉低 SCL 进行时钟拉伸；数据有效后，从机逐位驱动 SDA 发送给外部主机。

4.4 接口设计

4.4.1 I2C 接口

I2C 接口采用输入、输出和三态控制分离的形式。以主机模块为例，端口包括 `scl_i`、`scl_o`、`scl_t`、`sda_i`、`sda_o` 和 `sda_t`。在实际 FPGA 顶层中，通常将 `scl_t` 或 `sda_t` 作为是否释放总线的控制信号。当释放总线时，引脚呈高阻状态，由外部上拉电阻将信号拉高；当需要输出低电平时，模块拉低总线。

4.4.2 命令接口

主机命令接口包括目标地址、start、read、write、write_multiple 和 stop 等字段。命令接口使用 valid/ready 握手，保证命令只在双方均准备好时被接受。通过组合不同控制位，可表达读、写、多字节写和停止等事务。

4.4.3 数据接口

写数据和读数据均为 8 位数据宽度，并使用 AXI Stream 风格握手。`tvalid` 表示数据有效，`tready` 表示接收端准备好，`tlast` 表示多字节事务结束。该设计便于与 FIFO 和其他流式模块连接。

4.4.4 寄存器接口

AXI Lite 封装模块提供四类核心寄存器：状态寄存器、命令寄存器、数据寄存器和分频寄存器。状态寄存器反映 `busy`、`bus_control`、`bus_active`、`miss_ack`、FIFO 空满和溢出等状态；命令寄存器用于写入 I2C 地址和控制位；数据寄存器用于写入待发送数据或读取接收数据；分频寄存器用于配置 I2C 时钟。

5 系统实现

5.1 I2C 主机模块实现

`i2c_master` 是系统最关键的模块之一。其设计思想是将 I2C 事务分为命令控制和物理位传输两个层次。命令控制层负责解释上层输入的 `read`、`write`、`write multiple` 和 `stop` 等命令，判断是否需要产生 `start` 或 `repeated start`，管理地址、数据方向和事务结束条件。物理位传输层负责在 SCL 和 SDA 上产生符合 I2C 协议的位级时序。

主机模块定义了多个命令层状态，包括空闲、活动写、活动读、起始等待、起始、地址发送、写数据、读数据和停止等状态。空闲状态下，模块等待上层命令；当收到读写命令后，模块根据总线活动状态和地址变化判断是否需要发送起始条件；进入地址发送状态后，模块发送 7 位地址和读写位；随后根据命令类型进入读或写状态。

物理层状态机进一步细化 `start`、`repeated start`、写位、读位和 `stop` 的时序。写位时，模块先在 SCL 低电平期间设置 SDA，再拉高 SCL 使对方采样，随后拉低 SCL 完成一个时钟周期。读位时，模块释放 SDA，在 SCL 高电平期间采样 SDA 输入。`stop` 条件则要求在 SCL 高电平期间 SDA 由低变高。

主机模块还通过 `prescale` 控制 SCL 时钟周期。源码注释给出的计算方式为：

$$\text{prescale} = \text{Fclk} / (\text{FI2Cclk} * 4)$$

其中 `Fclk` 为系统输入时钟频率，`FI2Cclk` 为目标 I2C 时钟频率。通过分频计数器，模块能够在不同系统时钟下生成目标 I2C 速率。

5.2 I2C 从机模块实现

`i2c_slave` 模块用于响应外部 I2C 主机访问。该模块通过 `FILTER_LEN` 参数对 SCL 和 SDA 输入进行滤波，以降低毛刺对边沿检测的影响。模块通过比较当前和上一拍的 SCL/SDA 输入值识别 SCL 上升沿、SCL 下降沿、SDA 上升沿和 SDA 下降沿，并进一步判断 `start` 和 `stop` 条件。

从机在检测到 `start` 条件后进入地址接收状态，连续采样 7 位地址和 1 位读写方向。当接收到地址后，模块使用 `device_address` 与 `device_address_mask` 进行匹配。如果地址匹配，则输出 `ACK` 并进入读或写数据状态；如果地址不匹配，则释放总线并等待下一次事务。

在写方向下，外部主机向从机发送数据，从机逐位接收并组成字节。由于只有在下一个字节或 `stop` 条件出现时才能确定前一个字节是否为最后一个字节，因此模块

将 I2C 写入字节延迟一个字节时间后输出到 AXI Stream 接口。这样可以更准确地设置 `m_axis_data_tlast`。

在读方向下，从机需要向外部主机发送数据。若 AXI Stream 输入侧尚未提供有效数据，模块通过拉低 SCL 暂停主机时钟，实现时钟拉伸。待数据有效后，从机释放 SCL 并逐位输出数据。该机制保证从机不会在数据未准备好时发送错误内容。

5.3 AXI Lite 封装模块实现

`i2c_master_axil` 将 I2C 主机封装为 AXI Lite 从设备。该模块面向处理器控制场景，允许软件通过标准 AXI Lite 总线读写寄存器。模块参数包括默认分频、是否固定分频、命令 FIFO、写 FIFO、读 FIFO 及其深度等。

该模块的寄存器映射包括：

表 5.1 AXI Lite 封装寄存器

地址	寄存器	功能
0x00	Status	busy、bus_cont、 bus_act、miss_ack、 FIFO 状态
0x04	Command	I2C 地址和 start/read/write/write_multiple/stop 控制位
0x08	Data	写数据、读数据、 data_valid 和 data_last
0x0C	Prescale	I2C 分频配置

当处理器需要发起 I2C 写操作时，先写入数据寄存器或写 FIFO，再写命令寄存器发起 start/write/stop。需要读取外设时，处理器写命令寄存器发起读操作，再查询状态或数据寄存器获取返回数据。状态寄存器中的 `miss_ack` 可用于判断从设备是否响应，FIFO 状态位可用于避免上溢或下溢。

5.4 Wishbone 封装模块实现

Wishbone 封装模块与 AXI Lite 封装模块功能类似，但接口信号和寄存器组织方式适配 Wishbone 总线。`i2c_master_wbs_8` 使用 8 位数据总线，因此分频寄存器被拆分为低字节和高字节。其寄存器包括 Status、FIFO Status、Cmd Addr、Command、Data、Prescale Low 和 Prescale High。

Wishbone 封装的意义在于增强系统兼容性。许多开源 SoC 和软核处理器生态采用 Wishbone 作为片上外设总线，I2C 控制器通过 Wishbone 封装后，可以像普通外设一样被处理器读写。

5.5 I2C 初始化模块实现

`i2c_init` 是一个面向上电配置场景的模板模块。模块内部定义 `init_data` ROM 表，每个表项为 9 位命令字。命令可以表示停止、退出多设备模式、对当前地址开始写、开始地址块、开始数据块、延时、发送 I2C stop、对指定地址开始写以及写 8 位数据等操作。

模块启动后，状态机按顺序读取 ROM 表项，并将其转换为 I2C 主机命令接口和写数据接口输出。若使用单设备模式，ROM 表直接包含目标地址和写入数据；若使用多设备模式，ROM 表可先定义数据块，再定义多个设备地址，使同一初始化序列在多个设备上执行。

该模块适用于 FPGA 系统上电后自动配置外设。例如在没有 CPU 参与的系统中，可以通过修改 ROM 表，在启动阶段自动配置时钟芯片、PLL 或传感器寄存器。需要注意的是，当前源码中的 `init_data` 是示例数据，实际应用时必须根据目标外设的数据手册修改。

5.6 AXI Stream FIFO 实现

`axis_fifo` 是通用流式缓冲模块。其参数包括 FIFO 深度、数据宽度、是否启用 `tkeep`、是否启用 `tlast`、是否启用 `tid`、`tdest`、`tuser`、输出流水级数、帧 FIFO 模式和坏帧处理策略等。

FIFO 使用写指针和读指针管理存储空间。当写指针追上读指针的特定模式时，FIFO 判定为满；当写指针与读指针相等时，FIFO 判定为空。对于普通流式数据，`s_axis_tready` 在 FIFO 未满时有效；对于帧 FIFO 模式，还需要结合 `tlast` 判断帧边界。该模块可在 I2C 控制器封装中用于命令缓冲、写数据缓冲和读数据缓冲。

6 系统测试与结果分析

6.1 测试环境

项目 README 说明，运行测试平台需要 MyHDL、Icarus Verilog，并正确安装 `myhdl.vpi`。测试文件位于 `tb/` 目录，包括 Python 测试脚本和 Verilog 测试顶层。典型测试流程为：Python 脚本调用 Icarus Verilog 编译 RTL 与 Verilog testbench，生成 `.vvp` 文件；随后通过 `vvp -m myhdl` 加载 MyHDL VPI，使 Python 行为模型与 Verilog DUT 进行协同仿真。

以 `test_i2c_master.py` 为例，脚本中构造了 AXI Stream 命令源、写数据源和读数据接收端，并建立两个 I2C 存储器模型。其中一个存储器地址为 `0x50`，另一个地址为 `0x51`，且第二个模型设置了额外延迟，用于测试主机与不同响应速度从设备之间的通信。

6.2 测试内容设计

本文根据项目测试平台和模块功能，规划以下测试内容。

6.2.1 I2C 主机写测试

主机写测试用于验证主机模块能否正确发起写事务。测试步骤包括：向命令接口输入目标地址、start、write 和 stop 控制位；向写数据接口输入一个或多个字节；观察 SCL/SDA 是否产生正确起始条件、地址字段、写方向位、数据位、ACK 周期和停止条件；检查 busy 和 bus_active 状态是否随事务变化。

6.2.2 I2C 主机读测试

主机读测试用于验证主机模块能否从 I2C 从设备读取数据。测试步骤包括：配置 I2C 存储器模型中的数据；向主机命令接口输入读命令；检查主机是否发送地址和读方向位；检查读回数据是否通过 m_axis_data_tdata 输出；检查最后一个字节是否通过 m_axis_data_tlast 标记。

6.2.3 多字节写与 repeated start 测试

多字节写测试用于验证 write_multiple 和 tlast 机制。Repeated start 测试用于验证总线不释放情况下切换地址或方向的能力。例如典型寄存器读操作常先写入寄存器地址，再发送 repeated start 进入读方向。该测试能检验命令层状态机对连续事务的处理能力。

6.2.4 从机地址匹配测试

从机测试中，需要分别发送匹配地址和不匹配地址。地址匹配时，从机应输出 ACK 并进入读写状态；地址不匹配时，从机应释放总线，不应错误输出数据。通过修改 device_address_mask，还可测试部分地址匹配功能。

6.2.5 时钟拉伸测试

当外部主机读取从机，而从机本地输入数据尚未准备好时，从机应拉低 SCL 进行时钟拉伸。测试中可人为暂停 AXI Stream 输入侧数据，观察 SCL 是否被保持为低电平，并在数据有效后恢复传输。

6.2.6 AXI Lite 和 Wishbone 寄存器测试

寄存器测试用于验证总线封装模块的可用性。测试内容包括：读状态寄存器、写命令寄存器、写数据寄存器、读数据寄存器、配置分频寄存器、检查 FIFO 空满状态和溢出状态。对于 AXI Lite，需要验证读写地址、数据、响应通道是否符合协议；对于 Wishbone，需要验证 cyc、stb、we、ack 等信号配合是否正确。

6.3 当前验证结果

根据项目资料可以确认：

1. 项目为主要 RTL 模块提供了对应测试脚本和 Verilog 测试顶层，例如 test_i2c_master.py/.v、test_i2c_slave.py/.v、test_i2c_master_axil.py/.v、test_i2c_master_wbs_8.py/.v、test_i2c_slave_wbm.py/.v 等。

2. 测试平台包含 I2C 行为模型、AXI Stream 端点、AXI Lite BFM 和 Wishbone 模型，能够覆盖主从通信和总线封装场景。
3. 项目采用 MyHDL 与 Icarus Verilog 协同仿真，可通过 Python 构造更灵活的激励与检查逻辑。

由于当前写作阶段未在本机实际执行完整仿真命令，本文不虚构仿真通过率、波形截图、资源占用或最高频率等数据。后续正式定稿时应补充如下内容：

表 6.1 待补充测试结果记录表

测试项	预期结果	实际结果	备注
I2C 主机单字节写	产生 start、地址、写数据、ACK 和 stop	待补充	需附波形
I2C 主机多字节写	多字节连续传输，末字节由 tlast 标记	待补充	需附日志
I2C 主机读	读回数据与模型数据一致	待补充	需附波形
I2C 从机写	外部写入数据正确输出到 AXI Stream	待补充	需附检查结果
I2C 从机读	AXI Stream 输入数据正确发送到 I2C 总线	待补充	需验证 ACK/NACK
时钟拉伸	数据未准备好时 SCL 被拉低	待补充	需设置输入暂停
AXI Lite 寄存器访问	读写寄存器与状态位正确	待补充	需记录寄存器值
Wishbone 寄存器访问	Wishbone 读写握手正确	待补充	需记录总线波形

6.4 结果分析

从设计结构和测试平台完整性来看，该项目具备较好的可验证性。其一，主从核心均采用标准化接口，测试平台可以通过行为模型模拟外部 I2C 设备或主机，便于构造读写场景。其二，AXI Stream、AXI Lite 和 Wishbone 均有相应测试辅助模型，可降低测试代码编写难度。其三，Python 语言适合生成复杂激励和检查返回数据，因此 MyHDL 协同仿真比纯 Verilog testbench 更便于组织系统级场景。

但从毕业论文完整性角度看，仅有测试平台文件还不足以证明系统已经通过验证。正式论文应进一步补充仿真环境版本、运行命令、关键日志、波形截图和错误场景测试结果。若进行 FPGA 综合，还应补充目标芯片型号、综合工具版本、时钟约束、资源利用率和时序报告。若进行板级测试，还应补充外设型号、上拉电阻、I2C 速率、逻辑分析仪波形和读写数据记录。

7 总结与展望

7.1 全文总结

本文围绕 Verilog I2C interface 项目，完成了基于 Verilog 的 I2C 总线控制器设计与验证分析。论文首先介绍了 I2C 总线在 FPGA 外设控制中的应用背景，说明了硬件 I2C 控制器在板级配置、低速外设访问和 SoC 集成中的意义。随后，本文分析了 I2C 协议、Verilog HDL、FPGA、AXI Stream、AXI Lite、Wishbone 和协同仿真等相关技术，为后续系统设计奠定基础。

在需求分析部分，本文从主机通信、从机通信、总线封装、初始化序列和测试验证等方面归纳了系统功能需求，并总结了可综合性、可复用性、可配置性和可验证性等非功能需求。在系统设计部分，本文将项目划分为协议核心层、流式接口层、总线封装层和验证层，明确了各层之间的数据流和控制关系。

在系统实现部分，本文重点分析了 `i2c_master`、`i2c_slave`、`i2c_master_axil`、`i2c_master_wbs_8`、`i2c_init` 和 `axis_fifo` 等模块。主机模块采用命令层状态机与物理层状态机分层设计，从机模块通过地址匹配、输入滤波、数据转换和时钟拉伸实现 I2C 从设备功能，AXI Lite 和 Wishbone 封装模块通过寄存器方式提升系统集成能力，初始化模块则通过 ROM 表实现上电自动配置。

在测试验证部分，本文结合 MyHDL 与 Icarus Verilog 协同仿真平台，设计了主机写、主机读、多字节写、从机地址匹配、时钟拉伸、AXI Lite 寄存器访问和 Wishbone 寄存器访问等测试内容。由于当前材料尚未包含本机仿真和综合结果，论文未编造实验数据，而是明确列出需要补充的测试记录。

7.2 创新点与特点

本文所分析设计具有以下特点：

1. 模块化程度较高。项目将 I2C 主机、从机、FIFO、初始化逻辑、AXI Lite 封装和 Wishbone 封装分离，便于独立复用和维护。
2. 接口适配性较强。设计同时支持 AXI Stream、AXI Lite 和 Wishbone，能够适应纯逻辑控制、处理器寄存器控制和开源 SoC 集成等多种场景。
3. 状态机层次清晰。主机模块将事务级命令处理和位级物理时序分离，有利于降低复杂度。
4. 支持从机时钟拉伸。从机模块在本地数据未准备好时可通过拉低 SCL 暂停总线，提高通信可靠性。
5. 验证平台较完整。项目提供 MyHDL 行为模型和多个测试脚本，可覆盖主要模块和接口。

7.3 不足与展望

本文仍存在以下不足：

1. 未补充完整本机仿真日志和波形截图，测试结果部分仍需进一步实证化。
2. 未进行 FPGA 综合和时序分析，缺少 LUT、FF、BRAM、最高频率等硬件资源数据。
3. 未进行板级实物测试，无法给出真实外设访问结果和逻辑分析仪波形。

4. 中文参考文献条目仍需通过 CNKI 或学校图书馆进一步核验作者、来源、年份和页码。
5. 当前 `i2c_init` 中的初始化数据为模板示例，尚未结合具体外设手册形成真实应用案例。

后续工作可从以下方面展开：第一，搭建完整 MyHDL 与 Icarus Verilog 仿真环境，运行项目全部测试脚本并整理日志和波形；第二，选择目标 FPGA 平台进行综合和时序分析，评估资源占用与最高工作频率；第三，选取 EEPROM、温度传感器或时钟芯片等实际 I2C 外设进行板级验证；第四，补充异常场景测试，如 NACK、FIFO 溢出、总线占用、时钟拉伸和 repeated start；第五，结合具体 SoC 平台完善 AXI Lite 或 Wishbone 软件驱动示例。

参考文献

- [1] NXP Semiconductors. UM10204 I2C-bus specification and user manual[EB/OL]. 2021. 链接:<https://www.nxp.com/docs/en/user-guide/UM10204.pdf>. 获取方式：NXP 官方公开 PDF。
- [2] AMD/Xilinx. AXI IIC Bus Interface v2.1 LogiCORE IP Product Guide PG090[EB/OL]. 2021. 链接：<https://docs.amd.com/r/en-US/pg090-axi-iic>. 获取方式：AMD 官方文档。
- [3] OpenCores. I2C controller core[EB/OL]. 链接:<https://opencores.org/projects/i2c>. 获取方式：OpenCores 项目页面。
- [4] OpenCores. Wishbone B4 Specification[EB/OL]. 2010. 链接:https://cdn.opencores.org/downloads/wbspec_ 获取方式：OpenCores 官方规范文档。
- [5] Bergeron J. Writing Testbenches using SystemVerilog[M]. Boston: Springer, 2006. DOI: <https://doi.org/10.1007/0-387-25038-2>.
- [6] Spear C, Tumbush G. SystemVerilog for Verification: A Guide to Learning the Testbench Language Features[M]. New York: Springer, 2012. DOI: <https://doi.org/10.1007/978-1-4614-0715-7>.
- [7] ARM Limited. AMBA AXI and ACE Protocol Specification[EB/OL]. 获取方式：ARM 官方规范文档或授权文档渠道。注：最终定稿需补充具体版本号和链接。
- [8] IEEE. IEEE Standard for Verilog Hardware Description Language: IEEE Std 1364[S]. 获取方式：IEEE Xplore 或学校图书馆授权访问。注：最终定稿需补充具体版本和访问链接。
- [9] 【待核验】基于 FPGA 的 I2C 总线接口设计 [J/学位论文]. 注：中文参考文献需通过 CNKI 或学校图书馆核对作者、来源、年份、卷期和页码后替换。
- [10] 【待核验】基于 Verilog 的 I2C 控制器设计与实现 [J/学位论文]. 注：中文参考文献需通过 CNKI 或学校图书馆核对作者、来源、年份、卷期和页码后替换。

[11] 【待核验】I2C 总线协议的 FPGA 设计与仿真 [J/学位论文]. 注：中文参考文献需通过 CNKI 或学校图书馆核对作者、来源、年份、卷期和页码后替换。

[12] 【待核验】AXI 总线接口控制器的设计与实现 [J/学位论文]. 注：中文参考文献需通过 CNKI 或学校图书馆核对作者、来源、年份、卷期和页码后替换。

致谢

本论文的完成离不开指导教师课题方向、技术路线和论文写作方面的指导与帮助。在论文撰写过程中，老师对 I2C 协议分析、FPGA 硬件设计方法、模块化设计思路

和论文结构提出了宝贵意见，使本人能够更加系统地理解数字系统设计与验证流程。同时，感谢开源社区提供的 Verilog I2C interface 项目及相关技术资料。该项目为本文分析 I2C 主从控制器、AXI Lite/Wishbone 总线封装和 MyHDL 协同仿真提供了重要素材。感谢同学和朋友在资料查找、环境搭建和论文修改过程中给予的帮助。

最后，感谢家人在学习和毕业设计期间给予的支持与鼓励。由于本人能力和时间有限，论文中仍存在不足之处，恳请各位老师批评指正。

附录

附录 A 项目主要文件结构

```
verilog-i2c-master/  
  README.md  
  rtl/  
    axis_fifo.v  
    i2c_init.v  
    i2c_master.v  
    i2c_master_axil.v  
    i2c_master_wbs_8.v  
    i2c_master_wbs_16.v  
    i2c_single_reg.v  
    i2c_slave.v  
    i2c_slave_axil_master.v  
    i2c_slave_wbm.v  
  tb/  
    axil.py  
    axis_ep.py  
    i2c.py  
    wb.py  
    test_i2c_master.py  
    test_i2c_slave.py  
    test_i2c_master_axil.py  
    test_i2c_master_wbs_8.py
```

```
test_i2c_master_wbs_16.py
test_i2c_slave_axil_master.py
test_i2c_slave_wbm.py
```

附录 B 典型仿真命令

```
cd tb
python test_i2c_master.py
```

测试脚本内部典型编译命令如下：

```
iverilog -o test_i2c_master.vvp ../rtl/i2c_master.v test_i2c_master.v
vvp -m myhdl test_i2c_master.vvp -lxt2
```

附录 C 待补充材料清单

1. 学校模板中的封面、任务书和个人信息；
2. CNKI 已核验中文参考文献；
3. MyHDL/Icarus Verilog 仿真日志；
4. 关键 I2C 波形截图；
5. FPGA 综合资源报告；
6. 板级外设读写实验记录；
7. 最终 DOCX 或 LaTeX 模板排版结果。